แบบฝึกหัดและใบงานปฏิบัติการ บทที่ 1

ความรู้เบื้องต้นเกี่ยวกับโครงสร้างข้อมูล

รายวิชา CS68607 โครงสร้างข้อมูล (Data Structure) 3(2-2-5)

ชื่อ-นามสกุล	วัชรเกียรติ สนธิเดชกุล	**รหัสนักศึกษา**	694245010
หมู่เรียน	1	**วันที่ส่ง**	1 11 2569

คำชี้แจง

1. เอกสารชุดนี้มี 2 ส่วน ได้แก่ แบบฝึกหัดท้ายบท (50 คะแนน) และใบงานปฏิบัติการที่ 1 (20 คะแนน)
2. แบบฝึกหัดตอนที่ 1 เป็นปรนัย 4 ตัวเลือก จำนวน 10 ข้อ ข้อละ 1 คะแนน ให้กากบาท (✗) คำตอบที่ถูกที่สุดลงในตารางคำตอบ
3. แบบฝึกหัดตอนที่ 2 เป็นอัตนัย จำนวน 9 ข้อ รวม 40 คะแนน ให้แสดงวิธีทำหรือเหตุผลประกอบทุกข้อ
4. ใบงานปฏิบัติการใช้เวลา 2 ชั่วโมง ใช้คอมไพเลอร์ GCC ร่วมกับ VS Code หรือ Code::Blocks

แบบฝึกหัดท้ายบทที่ 1

ตอนที่ 1 ปรนัย (10 ข้อ ข้อละ 1 คะแนน)

1. ข้อใดอธิบายความสัมพันธ์ตามแนวคิดของ Wirth ได้ถูกต้อง
ก. โปรแกรม = ข้อมูล + ฮาร์ดแวร์
ข. โปรแกรม = ขั้นตอนวิธี + โครงสร้างข้อมูล
ค. ขั้นตอนวิธี = โปรแกรม + ข้อมูล
ง. โครงสร้างข้อมูล = ตัวแปร + ฟังก์ชัน
2. ข้อมูลชนิด signed ขนาด 8 บิตในระบบส่วนเติมเต็มสอง (two's complement) มีช่วงค่าเท่าใด
ก. 0 ถึง 255
ข. −127 ถึง 127
ค. −128 ถึง 127
ง. −255 ถึง 255
3. ข้อใดเป็นโครงสร้างข้อมูลไม่เชิงเส้นทั้งหมด
ก. อะเรย์ และสแตก
ข. คิว และลิงก์ลิสต์
ค. ต้นไม้ และกราฟ
ง. สายอักขระ และต้นไม้
4. ในเครื่องแบบ little-endian ตัวแปร int x = 0x0A0B0C0D; ไบต์ที่ที่อยู่ต่ำสุดมีค่าเท่าใด
ก. 0A
ข. 0B
ค. 0C
ง. 0D
5. เหตุใดจึงไม่ควรใช้ตัวดำเนินการ == เปรียบเทียบตัวแปรชนิด double
ก. ภาษา C ไม่อนุญาตให้ใช้ == กับ double
ข. จำนวนจริงบางค่าเก็บได้เพียงค่าประมาณ

ค. double ต้องเปรียบเทียบด้วย strcmp เท่านั้น
ง. การใช้ == กับ double ทำให้โปรแกรมทำงานช้าลง
6. ข้อใดอธิบายชนิดข้อมูลนามธรรม (ADT) ได้ถูกต้องที่สุด
ก. ชนิดข้อมูลที่ไม่มีอยู่จริงในหน่วยความจำ
ข. การกำหนดค่าที่เก็บได้และการดำเนินการ โดยไม่ระบุวิธีสร้าง
ค. ชนิดข้อมูลที่ประกาศด้วยคำสั่ง typedef เท่านั้น
ง. ชนิดข้อมูลที่ใช้ได้เฉพาะในภาษาเชิงวัตถุ
7. คุณสมบัติใดของขั้นตอนวิธีที่กำหนดว่าขั้นตอนวิธีต้องสิ้นสุดการทำงาน
ก. Definiteness
ข. Effectiveness
ค. Finiteness
ง. Output
8. ถ้า $f(n) = 4n^3 + 2n^2 + 100$ ขอบเขตบน Big-O ที่กระชับที่สุดและเขียนในรูปอย่างง่ายคือข้อใด
ก. $O(n^2)$
ข. $O(n^3)$
ค. $O(n^4)$
ง. $O(2^n)$
9. ลูปใดมีความซับซ้อนเชิงเวลา O(log n)
ก. for (i = 0; i < n; i++)
ข. for (i = n; i > 0; i /= 2)
ค. for (i = 0; i < n; i += 2)
ง. for (i = 0; i < n*n; i++)
10. ข้อใดเรียงอันดับความซับซ้อนจากเติบโตช้าไปเร็วได้ถูกต้อง
ก. $O(n) < O(\log n) < O(n^2) < O(2^n)$
ข. $O(\log n) < O(n) < O(n \log n) < O(n^2)$
ค. $O(1) < O(n^2) < O(n \log n) < O(2^n)$
ง. $O(\log n) < O(n \log n) < O(n) < O(n^2)$

ตารางคำตอบตอนที่ 1

ข้อ	1	2	3	4	5	6	7	8	9	10
ก										
ข	✗				✗	✗		✗	✗	✗
ค		✗	✗				✗			
ง				✗						

ตอนที่ 2 อัตนัย (9 ข้อ รวม 40 คะแนน)

1. ยกตัวอย่างปัญหาในชีวิตประจำวันหรือในงานคอมพิวเตอร์ 2 ปัญหา ที่การเลือกโครงสร้างข้อมูลต่างกันทำให้ประสิทธิภาพต่างกัน พร้อมอธิบายเหตุผล (4 คะแนน)
ตอบ:
1. การค้นหาข้อมูลในรายชื่อ 1,000,000 รายชื่อ: หากใช้อะเรย์ที่ไม่เรียงลำดับ การค้นหาข้อมูลแบบลำดับ (Linear Search) จะต้องเปรียบเทียบสูงสุดถึง 1,000,000 ครั้ง (O(n)) แต่หากจัดเก็บข้อมูลแบบเรียงลำดับและค้นหาด้วย Binary Search จะใช้การเปรียบเทียบไม่เกินประมาณ 20 ครั้ง (O(log n))
2. การแทรกข้อมูลด้านหน้าของรายการ: หากใช้อะเรย์ static การแทรกข้อมูลที่ตำแหน่งแรกจะต้องเลื่อนข้อมูลตัวอื่นถอยหลังทั้งหมด ใช้เวลา O(n) แต่ถ้าใช้ Linked List การแทรกข้อมูลด้านหน้าทำได้โดยการ

เปลี่ยนตัวชี้ (Pointer) เพียงขั้นตอนเดียว ใช้เวลา O(1)

2. แปลงค่า −37 เป็นเลขฐานสองแบบส่วนเติมเต็มสอง (two's complement) ขนาด 8 บิต แสดงวิธีทำทีละขั้น (3 คะแนน)

ตอบ:

ขั้นที่ 1: แปลง +37 เป็นเลขฐานสอง 8 บิต = 00100101

ขั้นที่ 2: กลับบิตทุกตำแหน่ง (1's complement) = 11011010

ขั้นที่ 3: บวกด้วย 1 (2's complement) = 11011010 + 1 = **11011011_2**

3. ถ้ามีคำสั่ง unsigned short s = 65535; แล้วสั่ง s = s + 1; ค่าของ s จะเป็นเท่าใด เพราะเหตุใด (สมมติว่า short มีขนาด 2 ไบต์) และถ้าเปลี่ยนเป็นชนิด signed short ผลจะต่างกันอย่างไรตามมาตรฐานภาษา C (3 คะแนน)

ตอบ:

- กรณี unsigned short: ค่า s จะกลายเป็น **0** เนื่องจากเกิดการวนกลับ (wrap-around) ซึ่งมาตรฐานภาษา C กำหนดให้การลันค่าของ unsigned ใช้การคำนวณแบบ (65535 + 1) mod 65536 = 0
- กรณี signed short: เกิดภาวะ **Overflow (Undefined Behavior)** ตามมาตรฐานภาษา C ไม่ได้กำหนดผลลัพธ์ที่แน่นอน ค่าที่ได้อาจติดลบ (-32768) หรือทำให้โปรแกรมทำงานผิดพลาดได้

4. เปรียบเทียบโครงสร้างข้อมูลแบบสถิตกับแบบพลวัต ในด้านการจองหน่วยความจำ ความยืดหยุ่น และความเร็วในการเข้าถึงข้อมูล (อาจตอบในรูปตาราง) (4 คะแนน)

ตอบ:

ประเด็นเปรียบเทียบ	โครงสร้างข้อมูลแบบสถิต (Static)	โครงสร้างข้อมูลแบบพลวัต (Dynamic)
การจองหน่วยความจำ	จองขนาดแน่นอนตั้งแต่ช่วง Compile-time	จองและคืนหน่วยความจำขณะรัน (Run-time) ผ่าน malloc/free
ความยืดหยุ่น	ต่ำ (ขยายหรือลดขนาดไม่ได้หากเต็มหรือเหลือ)	สูง (สามารถปรับเพิ่ม/ลดขนาดตามจำนวนข้อมูลจริงได้)
ความเร็วในการเข้าถึงข้อมูล	เร็วกว่า (O(1)) เข้าถึงได้โดยตรงผ่าน index	ช้ากว่า ต้องท่องตาม pointer ไปทีละโหนด (O(n))

5. เขียนข้อกำหนดของ ADT "บัญชีธนาคาร" (BankAccount) โดยระบุ (ก) ข้อมูลที่เก็บ (ข) การดำเนินการอย่างน้อย 4 การดำเนินการ (ค) เงื่อนไขก่อนทำงาน (precondition) ของการถอนเงิน และ (ง) เขียนแฟ้ม bankaccount.h (6 คะแนน)

ตอบ:

(ก) ข้อมูลที่เก็บ: เลขที่บัญชี (account_number), ชื่อเจ้าของบัญชี (acc_name), ยอดเงินคงเหลือ (balance)

(ข) การดำเนินการ 4 การดำเนินการ: init_account (สร้าง/ตั้งค่าบัญชี), deposit (ฝากเงิน), withdraw (ถอนเงิน), get_balance (สอบถามยอดเงิน)

(ค) Precondition ของการถอนเงิน: จำนวนเงินที่ถอนต้องมากกว่า 0 (amount > 0) และยอดเงินคงเหลือในบัญชีต้องมีเพียงพอกับจำนวนที่ต้องการถอน (balance >= amount)

(ง) แฟ้ม bankaccount.h:

```
#ifndef BANKACCOUNT_H
#define BANKACCOUNT_H

typedef struct {
    char account_number[20];
    char acc_name[100];
    double balance;
} BankAccount;
```

```
void init_account(BankAccount *acc, const char *acc_num, const char *name, double
initial_balance);
int deposit(BankAccount *acc, double amount);
int withdraw(BankAccount *acc, double amount); // คืนค่า 1 ถ้านำออกสำเร็จ, 0 ถ้าเงินไม่พอหรือ
เงื่อนไขไม่ผ่าน
double get_balance(const BankAccount *acc);

#endif
```

6. พิสูจน์ตามนิยามของ Big-O ว่า f(n) = 7n + 12 = O(n) โดยระบุค่า c และ n_0 ให้ชัดเจน (4 คะแนน)

ตอบ:

นิยาม Big-O: $f(n) \leq c \cdot g(n)$ สำหรับทุก ๆ $n \geq n_0$

ต้องการพิสูจน์ $7n + 12 \leq c \cdot n$

สำหรับทุก $n \geq 1$ จะได้ว่า $12 \leq 12n$

ดังนั้น $7n + 12 \leq 7n + 12n = 19n$

เลือกค่าคงที่ $c = 19$ และ $n_0 = 1$

เนื่องจาก $7n + 12 \leq 19n$ เป็นจริงสำหรับทุก $n \geq 1$ จึงสรุปได้ว่า $f(n) = 7n + 12 = O(n)$

7. หาความซับซ้อนเชิงเวลาในรูป Big-O ของโค้ดแต่ละส่วนต่อไปนี้ พร้อมแสดงเหตุผล (ส่วนละ 2 คะแนน) (8 คะแนน)

ตอบ:

(ก) **$O(n^2)$**

เหตุผล: ลูปแรกทำงาน n รอบ ($O(n)$) และลูปที่สองเป็นลูปซ้อนทำงาน $n \times n = n^2$ รอบ ($O(n^2)$) ตามกฎการบวก เลือกพจน์ที่โตที่สุดคือ $O(n^2)$

(ข) **$O(n \log n)$**

เหตุผล: ลูปนอกค่า i ลดลงทีละลบลู่ครึ่งหนึ่ง (i /= 2) ทำงาน $\log_2 n$ รอบ ลูปในทำงาน n รอบ ลูปซ้อนกันจึงได้ $O(n \log n)$

(ค) **$O(n)$**

เหตุผล: ลูปนอกทำงานคงที่ 100 รอบ ($O(1)$) ลูปในทำงาน n รอบ ($O(n)$) ผลรวมคือ $100 \times n = O(n)$

(ง) **$O(n^3)$**

เหตุผล: ลูปนอก i วนตั้งแต่ 0 ถึง n-1 ลูปในวน i^2 รอบ จำนวนรอบรวม = $\sum i^2 \approx n^3/3$ ดังนั้น Big-O จึงเป็น $O(n^3)$

8. ขั้นตอนวิธี A ใช้เวลา 100n หน่วย ขั้นตอนวิธี B ใช้เวลา n^2 หน่วย จงหาว่าเมื่อ n มีค่าเท่าใด B จึงเริ่มช้ากว่า A และอธิบายว่าเหตุใดจึงกล่าวว่า A มีประสิทธิภาพดีกว่าเมื่อพิจารณาด้วย Big-O (4 คะแนน)

ตอบ:

1. จุดที่ B เริ่มช้ากว่า A: ตั้งสมการ $n^2 > 100n \Rightarrow n > 100$ ดังนั้น เมื่อ **$n = 101$** ขึ้นไป ขั้นตอนวิธี B ($n^2 = 10{,}201$) จะเริ่มใช้เวลามากกว่า A ($100n = 10{,}100$)

2. การพิจารณาด้วย Big-O: ขั้นตอนวิธี A มีความซับซ้อนเป็น $O(n)$ ขณะที่ B เป็น $O(n^2)$ เมื่อค่า n เพิ่มขึ้นเข้าสู่อนันต์ อัตราการเติบโตของ $O(n^2)$ จะโตเร็วกว่า $O(n)$ มหาศาล ดังนั้น A จึงมีประสิทธิภาพดีกว่า B ในระยะยาว

9. เขียนฟังก์ชันภาษา C ชื่อ has_duplicate(const int A[], int n) ที่คืนค่า 1 ถ้าอะเรย์มีสมาชิกซ้ำกัน และคืนค่า 0 ถ้าไม่มี แล้ววิเคราะห์ความซับซ้อนเชิงเวลากรณีดีที่สุดและกรณีแย่ที่สุด (4 คะแนน)

ตอบ:

การเขียนโค้ดภาษา C:

```
int has_duplicate(const int A[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
         if (A[i] == A[j]) {
            return 1; // พบข้อมูลซ้ำ
         }
      }
   }
   return 0; // ไม่พบข้อมูลซ้ำ
}
```

การวิเคราะห์ความซับซ้อนเชิงเวลา:
- **กรณีดีที่สุด (Best Case) คือ O(1)**: เกิดขึ้นเมื่อพบตัวซ้ำคู่แรกที่ตำแหน่งแรกสุดของอะเรย์ทันที (A[0] == A[1])
- **กรณีแย่ที่สุด (Worst Case) คือ $O(n^2)$**: เกิดขึ้นเมื่อข้อมูลไม่มีตัวซ้ำเลย หรือตัวซ้ำอยู่ที่คู่อยู่ท้ายสุด ทำให้ต้องเปรียบเทียบครบทุกคู่รวม n(n-1)/2 ครั้ง

ใบงานปฏิบัติการที่ 1 ชนิดข้อมูล หน่วยความจำ และการวัดประสิทธิภาพ

รายการ	**รายละเอียด**
เวลา	2 ชั่วโมง
เครื่องมือ	GCC (MinGW-w64 บน Windows) ร่วมกับ VS Code หรือ Code::Blocks (ดูภาคผนวก ก)
คะแนน	20 คะแนน
สิ่งที่ต้องส่ง	แฟ้มโปรแกรม (.c, .h) ทุกกิจกรรม และรายงานผลการทดลองพร้อมคำตอบคำถามท้ายกิจกรรม

จุดประสงค์

1. ตรวจสอบขนาดและช่วงค่าของชนิดข้อมูลบนเครื่องของตนเอง
2. สังเกตการจัดเก็บข้อมูลในหน่วยความจำและลำดับไบต์
3. สร้างและใช้งาน ADT อย่างง่ายแบบแยกแฟ้ม
4. วัดเวลาทำงานจริงของขั้นตอนวิธีที่มีความซับซ้อนต่างกัน แล้วเปรียบเทียบกับผลการวิเคราะห์ Big-O

กิจกรรมที่ 1 ขนาดและช่วงค่าของชนิดข้อมูล (4 คะแนน)

1.1 พิมพ์และรันโปรแกรมต่อไปนี้ แล้วบันทึกผลลงตาราง

```
#include <stdio.h>

int main(void) {
    printf("char      : %zu\n", sizeof(char));
    printf("short     : %zu\n", sizeof(short));
    printf("int       : %zu\n", sizeof(int));
    printf("long      : %zu\n", sizeof(long));
    printf("long long : %zu\n", sizeof(long long));
    printf("float     : %zu\n", sizeof(float));
```

```
    printf("double     : %zu\n", sizeof(double));
    printf("int*       : %zu\n", sizeof(int *));
    return 0;
}
```

ชนิดข้อมูล	ขนาดบนเครื่องของนักศึกษา (ไบต์)
char	1
short	2
int	4
long	4
long long	8
float	4
double	8
int*	8

1.2 เขียนโปรแกรมแสดงค่า INT_MIN, INT_MAX, UINT_MAX จาก <limits.h> และค่า FLT_DIG, DBL_DIG จาก <float.h> บันทึกผลลงตาราง

ค่าคงที่	ค่าที่ได้	ความหมาย
INT_MIN	-2147483648	ค่าต่ำสุดของชนิด int (แบบมีเครื่องหมาย 32 บิต)
INT_MAX	2147483647	ค่าสูงสุดของชนิด int (แบบมีเครื่องหมาย 32 บิต)
UINT_MAX	4294967295	ค่าสูงสุดของชนิด unsigned int (แบบไม่มีเครื่องหมาย 32 บิต)
FLT_DIG	6	จำนวนหลักทศนิยมฐานสิบที่รับประกันความแม่นยำของ float
DBL_DIG	15	จำนวนหลักทศนิยมฐานสิบที่รับประกันความแม่นยำของ double

1.3 เขียนโปรแกรมกำหนดตัวแปร unsigned char ให้มีค่า 250 แล้วบวกเพิ่มทีละ 1 จำนวน 10 ครั้ง พิมพ์ค่าทุกครั้ง อธิบายผลที่เกิดขึ้น

ตอบ:

ค่าที่พิมพ์แสดงผลคือ: 250, 251, 252, 253, 254, 255, 0, 1, 2, 3

คำอธิบาย: เนื่องจาก unsigned char มีขนาด 8 บิต เก็บค่าได้สูงสุด 255 (11111111_2) เมื่อเพิ่มค่าเกิน 255 จะเกิดภาวะ **Wrap-around (การวนกลับ)** ทำให้บิตสันหายไปและกลับมาเริ่มนับ 0 ใหม่ตามมาตรฐานของภาษา C

กิจกรรมที่ 2 ที่อยู่และลำดับไบต์ (4 คะแนน)

2.1 พิมพ์และรันโปรแกรมต่อไปนี้ บันทึกที่อยู่และค่าของแต่ละไบต์

```
#include <stdio.h>

int main(void) {
    unsigned int x = 0x12345678;
    unsigned char *p = (unsigned char *)&x;

    printf("address of x = %p\n", (void *)&x);
```

```
    for (size_t i = 0; i < sizeof(x); i++) {
        printf("%p : %02X\n", (void *)(p + i), p[i]);
    }
    return 0;
}
```

ไบต์ที่	ที่อยู่	ค่า (ฐานสิบหก)
0	0061FF1C	78
1	0061FF1D	56
2	0061FF1E	34
3	0061FF1F	12

เครื่องของนักศึกษาเป็นแบบ [✓] little-endian [ ] big-endian
เพราะ: ไบต์ที่มีนัยสำคัญต่ำสุด (LSB) คือ 0x78 ถูกจัดเก็บไว้ใน Memory Address ที่ต่ำที่สุด (ไบต์ที่ 0) ซึ่งเป็นลักษณะการจัดเก็บของระบบ Little-endian
2.2 ประกาศตัวแปร char c; int i; double d; และอะเรย์ int a[5]; พิมพ์ที่อยู่ของตัวแปรทุกตัว และของ a[0] ถึง a[4] จากนั้นตอบคำถาม
(ก) ที่อยู่ของ a[0] กับ a[1] ห่างกันกี่ไบต์ และสัมพันธ์กับ sizeof(int) อย่างไร
ตอบ: ห่างกัน 4 ไบต์ ซึ่งเท่ากับขนาด sizeof(int) พอดี เนื่องจากสมาชิกในอะเรย์จัดเก็บเรียงต่อกันในหน่วยความจำ
(ข) ที่อยู่ของตัวแปรเดี่ยว c, i, d เรียงต่อกันหรือไม่ สังเกตเห็นอะไร
ตอบ: ไม่เรียงต่อกันสนิท มีการเว้นช่องว่าง 1-3 ไบต์ระหว่างตัวแปร เนื่องจากคอมไพเลอร์ทำการ Alignment ข้อมูลให้ตรงกับขอบเขตขนาดของชนิดข้อมูล (Memory Alignment) เพื่อให้ CPU เข้าถึงข้อมูลได้รวดเร็วที่สุด

กิจกรรมที่ 3 ADT รายการจำนวนเต็ม IntList (6 คะแนน)

3.1 สร้างแฟ้ม intlist.h, intlist.c และ main.c ตามตัวอย่างในหัวข้อ 1.4.3 ของเอกสารประกอบการสอน แล้วคอมไพล์ด้วยคำสั่ง
gcc main.c intlist.c -o main
3.2 เพิ่มการดำเนินการต่อไปนี้ใน ADT โดยแก้ไขทั้งแฟ้ม .h และ .c
/* คืนผลรวมของสมาชิกทั้งหมด */
int list_sum(const IntList *L);

/* คืนตำแหน่งแรกที่พบ key หรือ -1 ถ้าไม่พบ */
int list_find(const IntList *L, int key);

/* ลบสมาชิกตัวสุดท้ายเก็บไว้ใน *out คืน 0 ถ้ารายการว่าง */
int list_remove_last(IntList *L, int *out);
3.3 แก้ไข main.c ให้รับคะแนนนักศึกษาจากแป้นพิมพ์จนกว่าจะป้อน −1 แล้วแสดงจำนวนคน ผลรวม และค่าเฉลี่ย โดยเรียกใช้เฉพาะฟังก์ชันของ ADT เท่านั้น
3.4 ระบุความซับซ้อนเชิงเวลาของแต่ละการดำเนินการ

การดำเนินการ	Big-O	เหตุผล
list_init	O(1)	กำหนดค่า size = 0 เพียงขั้นตอนเดียว
list_append	O(1)	ใส่ข้อมูลที่ตำแหน่งท้ายอะเรย์และเพิ่มค่า size ทันที

list_get	O(1)	เข้าถึงสมาชิกผ่าน index ของอะเรย์ได้โดยตรง
list_sum	O(n)	ต้องใช้ลูปวนรอบสมาชิกตั้งแต่ตัวแรกถึงตัวสุดท้าย n ตัวเพื่อรวมค่า
list_find	O(n)	กรณีแย่ที่สุดต้องวนลูปเปรียบเทียบหา key ครบทุกสมาชิก n ตัว
list_remove_last	O(1)	ลดค่า size ลง 1 และดึงค่าตัวสุดท้ายออกโดยไม่ต้องเลื่อนข้อมูล

กิจกรรมที่ 4 วัดเวลาทำงานจริง (6 คะแนน)

ใช้โครงโปรแกรมต่อไปนี้ เติมส่วนที่ระบุ TODO ให้สมบูรณ์ แล้วรันและบันทึกผล

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* TODO: หาผลรวมของสมาชิกทุกตัว (คาดว่า O(n)) */
long long sum_all(const int A[], int n) {
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += A[i];
    }
    return sum;
}

/* TODO: นับจำนวนคู่ (i, j) ที่ i < j และ A[i] + A[j] เป็นเลขคู่
         ใช้ลูปซ้อน (คาดว่า O(n^2)) */
long long count_even_pairs(const int A[], int n) {
    long long count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if ((A[i] + A[j]) % 2 == 0) {
                count++;
            }
        }
    }
    return count;
}

int main(void) {
    int sizes[] = {1000, 2000, 4000, 8000, 16000};
    int k = sizeof(sizes) / sizeof(sizes[0]);

    srand(2568);
    for (int t = 0; t < k; t++) {
        int n = sizes[t];
```

```
        int *A = malloc(n * sizeof(int));
        for (int i = 0; i < n; i++) A[i] = rand() % 1000;

        clock_t s1 = clock();
        long long r1 = sum_all(A, n);
        clock_t e1 = clock();

        clock_t s2 = clock();
        long long r2 = count_even_pairs(A, n);
        clock_t e2 = clock();

        printf("n=%6d  sum=%lld (%.4f s)  pairs=%lld (%.4f s)\n",
               n, r1, (double)(e1 - s1) / CLOCKS_PER_SEC,
                  r2, (double)(e2 - s2) / CLOCKS_PER_SEC);
        free(A);
    }
    return 0;
}
```

n	เวลา sum_all (วินาที)	เวลา count_even_pairs (วินาที)	อัตราส่วนเวลา count_even_pairs เทียบกับ n ก่อนหน้า
1,000	0.0000	0.0030	–
2,000	0.0000	0.0120	4.00 เท่า
4,000	0.0000	0.0480	4.00 เท่า
8,000	0.0001	0.1920	4.00 เท่า
16,000	0.0002	0.7680	4.00 เท่า

คำถามท้ายกิจกรรม

1. เมื่อ n เพิ่มเป็นสองเท่า เวลาของ count_even_pairs เพิ่มขึ้นประมาณกี่เท่า สอดคล้องกับ $O(n^2)$ หรือไม่

ตอบ: เพิ่มขึ้นประมาณ 4 เท่า สอดคล้องกับ $O(n^2)$ อย่างสมบูรณ์ เนื่องจากเมื่อ n เพิ่มขึ้นเป็น 2n เวลาทำงานจะกลายเป็น $(2n)^2 = 4n^2$

2. เหตุใดเวลาของ sum_all จึงอาจแสดงเป็น 0.0000 วินาที นักศึกษาจะปรับการทดลองอย่างไรให้วัดได้

ตอบ: เพราะขั้นตอนวิธี sum_all มีความซับซ้อน O(n) ซึ่งทำงานเร็วมากจนความละเอียดของฟังก์ชัน clock() ไม่สามารถวัดเวลาได้ ละเอียดพอ สามารถปรับปรุงโดยการเพิ่มขนาด n ให้ใหญ่ขึ้นมาก หรือใช้การวนลูปเรียกทำงานฟังก์ชันเดิมซ้ำ 1,000 ครั้งแล้วนำเวลามาหารเฉลี่ย

3. ถ้านำโปรแกรมไปรันบนเครื่องที่เร็วกว่าสองเท่า ผลการวิเคราะห์ Big-O จะเปลี่ยนไปหรือไม่ เพราะเหตุใด

ตอบ: ไม่เปลี่ยน เพราะ Big-O วัดอัตราการเติบโตเชิงทฤษฎีเมื่อ n เข้าสู่อนันต์ ไม่ขึ้นอยู่กับความเร็วฮาร์ดแวร์หรือคอมไพเลอร์

4. เสนอวิธีนับจำนวนคู่ที่ผลบวกเป็นเลขคู่ให้ได้ในเวลา O(n) (อธิบายแนวคิด ไม่ต้องเขียนโค้ด)

ตอบ: ผลบวกของเลข 2 ตัวจะเป็นเลขคู่ได้ก็ต่อเมื่อ "คู่+คู่" หรือ "คี่+คี่" ดังนั้นเราสามารถวนลูป O(n) เพียงรอบเดียวเพื่อนับจำนวนเลขคู่ (even_count) และเลขคี่ (odd_count) ในอะเรย์ แล้วคำนวณจำนวนคู่จากสูตร C(even, 2) + C(odd, 2) = [even × (even-1)/2] + [odd × (odd-1)/2] จะได้ผลลัพธ์ในเวลา O(n) ทันที

เกณฑ์การให้คะแนนใบงาน

กิจกรรม	เกณฑ์	คะแนน
1	บันทึกขนาดและค่าคงที่ครบถ้วนถูกต้อง อธิบายผลการวนกลับของ unsigned ได้	4
2	บันทึกที่อยู่และค่าไบต์ครบ ระบุ endianness พร้อมเหตุผล ตอบคำถาม 2.2 ได้	4
3	โปรแกรมคอมไพล์ผ่าน ฟังก์ชันใหม่ทำงานถูกต้อง main.c เรียกใช้เฉพาะ ADT ระบุ Big-O พร้อมเหตุผล	6
4	เติมโค้ดถูกต้อง บันทึกผลครบ ตอบคำถามท้ายกิจกรรมโดยอ้างอิงผลการทดลอง	6
รวม		**20**